Universidad de San Carlos de Guatemala

Lenguajes Formales y de Programación
Sección B+ · Segundo semestre 2026

Manual Técnico de MarkdownCheck

José Eliú Castro Sosa
Carné 202405110

Índice

1. De qué trata este manual 3

2. Cómo está armado el programa 3

2.1 El camino que sigue un archivo 3

2.2 Las carpetas 4

2.3 Las clases y qué hace cada una 4

3. El analizador léxico 4

3.1 Qué hace y cómo avanza 4

3.2 Los tokens 5

3.3 Las tres confusiones y cómo las resuelvo 5

3.4 La sangría de las listas 6

3.5 Los bloques de código 6

3.6 Cómo decide el lector, visto como tabla de estados 6

4. La gramática 7

4.1 Qué ajusté de la gramática original y por qué 8

4.2 Un conflicto que quedó y cómo lo resuelvo 9

5. El analizador sintáctico 10

5.1 La idea general 10

5.2 Cómo decide qué hacer 10

5.3 La recursión en las listas 10

5.4 La recursión en las citas 10

5.5 El texto dentro de una línea 11

6. El árbol 11

7. Errores y cómo se recupera el programa 12

7.1 Los siete errores 12

7.2 La estrategia de recuperación 12

7.3 Un ejemplo de cadena inválida 12

8. Los reportes 13

9. Decisiones de diseño y por qué las tomé 13

10. Pruebas y límites 14

1. De qué trata este manual

MarkdownCheck es un programa hecho en Python que lee un archivo de texto escrito en Markdown, revisa que esté bien escrito y lo convierte en una página web. Si el archivo trae errores, el programa los anota todos con su línea y su columna, sigue revisando hasta el final y al terminar los muestra.

Este manual explica cómo está construido por dentro: qué partes tiene, cómo se pasan la información entre ellas, qué reglas sigue para entender el texto y qué decisiones tomé y por qué. Está escrito para que alguien que sepa programar un poco lo pueda seguir sin conocer la teoría a fondo.

Hay dos reglas del proyecto que condicionan todo lo demás. La primera es que no se puede usar el módulo de expresiones regulares para leer el texto, así que todo se recorre letra por letra a mano. La segunda es que no se puede usar ninguna librería que haga el análisis ni que convierta Markdown, así que tanto el lector como el parser están escritos desde cero. Además, el HTML no se arma cambiando pedazos de texto, sino recorriendo un árbol que el parser construye.

2. Cómo está armado el programa

2.1 El camino que sigue un archivo

Todo funciona como una línea de producción. El archivo entra por un lado y va pasando por cuatro etapas hasta salir convertido en reportes.

1. La interfaz gráfica carga el archivo y se lo entrega al analizador léxico.
2. El analizador léxico lee el texto letra por letra y lo parte en piezas pequeñas llamadas tokens. Cada token es algo como un símbolo de encabezado, una viñeta, un pedazo de texto o un delimitador de negrita.
3. El analizador sintáctico recibe esos tokens y revisa que vengan en un orden válido. Mientras revisa, va construyendo un árbol que representa la estructura del documento.
4. Los generadores recorren ese árbol y producen los reportes y el archivo del árbol de derivación.

En paralelo trabaja el gestor de errores, que recibe los problemas que encuentren el lector y el parser y los guarda en una lista para mostrarlos al final.

2.2 Las carpetas

El proyecto está ordenado así:

Carpeta	Qué hay adentro
src	El código del programa: lector, parser, árbol, interfaz y generadores.
ejemplos	Archivos de Markdown para probar el programa, trece en total.
tests	Los scripts de prueba y el generador de los casos de prueba.
docs	La documentación, incluido el documento de casos de prueba.
reportes	Aquí se guardan los archivos que genera el programa al analizar.

El programa se arranca desde la carpeta del proyecto con el comando `python main.py`, que abre la interfaz gráfica. Necesita Python 3.10 o más nuevo y usa Tkinter, que ya viene con Python.

2.3 Las clases y qué hace cada una

Clase	Para qué sirve
Token	Guarda una pieza del texto: su tipo, el texto que la formó, y la línea y columna donde estaba.
AnalizadorLexico	Recorre el archivo y entrega un token cada vez que se le pide con el método siguiente_token.
NodoAST	Es un nodo del árbol. Guarda su tipo, su contenido y una lista con sus hijos.
AnalizadorSintactico	Es el parser. Pide tokens, revisa el orden y construye el árbol.
GestorErrores	Junta todos los errores con su tipo, su descripción y su posición.
GeneradorHTML	Recorre el árbol y escribe el documento HTML, las estadísticas y la tabla de contenido.
Clase de la interfaz	Maneja la ventana, los botones y las tablas de tokens y errores.

La idea de fondo es que cada clase hace una sola cosa. El lector no sabe nada de HTML y el generador no sabe nada de cómo se leyó el archivo. Solo se comunican a través de los tokens y del árbol.

3. El analizador léxico

3.1 Qué hace y cómo avanza

El analizador léxico lleva tres cosas anotadas todo el tiempo: en qué posición del texto va, en qué línea está y en qué columna. Cada vez que avanza una letra actualiza esos contadores. Gracias a eso cada token sabe exactamente dónde nació, y de ahí salen las posiciones que se muestran en las tablas.

Para decidir qué token sigue, mira la letra actual y, si hace falta, la que viene después. Ese proceso de decidir mirando letra por letra es lo que se conoce como autómata finito. En mi

programa no está guardado como una tabla de estados, sino escrito como una serie de decisiones en el código. Más abajo está descrito como si fuera una tabla para que se pueda estudiar.

3.2 Los tokens

Token	Qué es
NUMERAL	De uno a seis símbolos # al inicio de línea, seguidos de un espacio. Marca un encabezado.
VIÑETA	Un guion, un asterisco o un más al inicio de línea, después de la sangría, y seguido de un espacio. Marca un ítem de lista.
INDENT	Token invisible. Aparece cuando un ítem queda un nivel más adentro que el anterior.
DEDENT	Token invisible. Aparece cuando se sale de un nivel. Si se suben dos niveles de golpe salen dos.
CITA	Un símbolo > al inicio de línea. Un >> son dos tokens seguidos y un > > da exactamente lo mismo.
CERCA_CODIGO	Tres acentos graves al inicio de línea. Abre y cierra un bloque de código.
LENGUAJE	La palabra pegada a la cerca de apertura, como python. Se usa para marcar el lenguaje en el HTML.
TEXTO_LITERAL	Todo lo que hay dentro de un bloque de código o de un código en línea, tal cual, en un solo token.
REGLA_HORIZONTAL	Tres o más guiones solos en una línea.
NEGRITA	Doble asterisco. Abre o cierra una negrita.
CURSIVA	Un asterisco que no está en posición de viñeta. Abre o cierra una cursiva.
CODIGO_INLINE	Un acento grave simple. Abre o cierra un código dentro de una línea.
TEXTO	Texto normal que no es ninguno de los símbolos anteriores.
SALTO_PARRAFO	Una línea en blanco. Ella sola es el token completo.
FIN_LINEA	El final de una línea que sí tiene contenido.
EOF	El final del archivo.

Los tres últimos tipos, FIN_LINEA, EOF y LENGUAJE, los agregué sobre la lista original del enunciado, igual que TEXTO_LITERAL, que ya aparecía en la gramática pero no en la tabla. FIN_LINEA hace falta para que el parser sepa dónde termina un encabezado, un ítem o una línea de párrafo. EOF le dice cuándo parar. LENGUAJE sirve para poder escribir la clase del bloque de código en el HTML.

3.3 Las tres confusiones y cómo las resuelvo

Markdown usa los mismos símbolos para cosas distintas, así que el lector tiene reglas de desempate.

Viñeta o cursiva. Un guion, un asterisco o un más solo cuentan como viñeta si están al inicio de la línea, después de la sangría, y tienen un espacio justo después. Si el asterisco aparece en cualquier otro lugar, empieza una cursiva.

Negrita o cursiva. Cuando el lector encuentra un asterisco mira si el siguiente también lo es. Si lo es, toma los dos juntos como una negrita. Siempre se queda con la opción más larga, y por eso dos asteriscos nunca se leen como dos cursivas.

Encabezado o texto. Los símbolos # solo forman un encabezado si están al inicio de línea y llevan un espacio después. Si no, como en #sinespacio, es texto normal.

3.4 La sangría de las listas

Cada nivel de una lista equivale a dos espacios más que el nivel que lo contiene. Para llevar la cuenta el lector usa una pila de números donde guarda las sangrías por las que va pasando.

- Si una línea tiene más sangría que la anterior, apila el nuevo nivel y entrega un INDENT.
- Si tiene menos, desapila todos los niveles que haga falta y entrega un DEDENT por cada uno.
- Si tiene la misma, no entrega nada especial.

Si la sangría crece más de un nivel de golpe, por ejemplo de cero a cuatro espacios sin pasar por dos, el lector lo reporta como indentación inválida.

3.5 Los bloques de código

Cuando el lector encuentra la cerca de apertura entra en lo que llamo modo literal. Desde ahí deja de buscar negritas, cursivas, viñetas o cualquier otro símbolo y junta todo lo que encuentra en un solo TEXTO_LITERAL hasta toparse con la cerca de cierre. Por eso un código como x = 2 ** 3 no se confunde con una negrita.

Lo mismo pasa dentro de un código en línea: lo que queda entre los dos acentos graves se toma tal cual.

3.6 Cómo decide el lector, visto como tabla de estados

Esta tabla describe las decisiones que toma el lector, empezando siempre en el inicio de una línea. Es una explicación del comportamiento, no una tabla que exista en el código.

Estado	Qué ve	A dónde pasa
Inicio de línea	Una letra #	Cuenta los # seguidos. Si son de uno a seis y hay un espacio, entrega NUMERAL. Si son más de seis, reporta encabezado inválido.
Inicio de línea	Un guion, asterisco o más con espacio después	Entrega VIÑETA y pasa a leer contenido de línea.
Inicio de línea	Tres guiones solos	Entrega REGLA_HORIZONTAL.
Inicio de línea	Un símbolo >	Entrega CITA y revisa si sigue otro > o un espacio.
Inicio de línea	Tres acentos graves	Entrega CERCA_CODIGO y entra en modo literal.

Estado	Qué ve	A dónde pasa
Inicio de línea	Línea vacía	Entrega SALTO_PARRAFO.
Contenido de línea	Dos asteriscos	Entrega NEGRITA.
Contenido de línea	Un asterisco	Entrega CURSIVA.
Contenido de línea	Un acento grave	Entrega CODIGO_INLINE y lee el literal hasta el siguiente.
Contenido de línea	Cualquier otra letra	La suma al texto actual y, al llegar a un símbolo, entrega TEXTO.
Contenido de línea	Salto de línea	Entrega FIN_LINEA y vuelve al inicio de línea.
Modo literal	Cualquier cosa menos la cerca	La junta en un solo TEXTO_LITERAL.
Modo literal	Cerca de cierre	Entrega CERCA_CODIGO y sale del modo literal.

4. La gramática

La gramática es el conjunto de reglas que dice qué órdenes de tokens son válidos. Parto de la que da el enunciado y la ajusté para que un parser que baja recursivamente pueda decidir sin adivinar. Los nombres entre signos de menor y mayor son partes de la estructura, y los nombres en mayúsculas son tokens. El símbolo inicial es documento. La letra ε significa que esa parte puede quedar vacía.

```
<documento> ::= <bloques> EOF

<bloques> ::= <bloque> <bloques>
            | ε

<bloque> ::= <encabezado>
           | <parrafo>
           | <lista>
           | <cita>
           | <bloque_codigo>
           | <regla>
           | SALTO_PARRAFO

<encabezado> ::= NUMERAL <en_linea> FIN_LINEA

<regla> ::= REGLA_HORIZONTAL FIN_LINEA

<parrafo> ::= <linea_texto> <parrafo_resto>

<parrafo_resto> ::= <linea_texto> <parrafo_resto>
                  | ε

<linea_texto> ::= <elemento> <en_linea> FIN_LINEA

<lista> ::= <item_lista> <lista_resto>

<lista_resto> ::= <item_lista> <lista_resto>
                | ε

<item_lista> ::= VIÑETA <en_linea> FIN_LINEA <sublista_opt>
```

```
<sublista_opt> ::= INDENT <lista> DEDENT
                 | ε

<cita> ::= <linea_cita> <cita_resto>

<cita_resto> ::= <linea_cita> <cita_resto>
               | ε

<linea_cita> ::= CITA <cuerpo_cita>

<cuerpo_cita> ::= CITA <cuerpo_cita>
                | <en_linea> FIN_LINEA

<bloque_codigo> ::= CERCA_CODIGO <lenguaje_opt> FIN_LINEA
                    <contenido_cod> CERCA_CODIGO FIN_LINEA

<lenguaje_opt> ::= LENGUAJE
                 | ε

<contenido_cod> ::= TEXTO_LITERAL
                  | ε

<en_linea> ::= <elemento> <en_linea>
             | ε

<elemento> ::= <negrita>
             | <cursiva>
             | <codigo_inline>
             | TEXTO

<negrita> ::= NEGRITA <en_negrita> NEGRITA

<en_negrita> ::= <elem_neg> <en_negrita>
               | ε

<elem_neg> ::= <cursiva>
             | <codigo_inline>
             | TEXTO

<cursiva> ::= CURSIVA <en_cursiva> CURSIVA

<en_cursiva> ::= <elem_curs> <en_cursiva>
               | ε

<elem_curs> ::= <negrita>
              | <codigo_inline>
              | TEXTO

<codigo_inline> ::= CODIGO_INLINE TEXTO_LITERAL CODIGO_INLINE
```

4.1 Qué ajusté de la gramática original y por qué

Un parser que baja recursivamente decide qué regla usar mirando solo el token que viene. Para que eso funcione, la gramática no puede tener reglas que empiecen llamándose a sí mismas ni dos opciones que empiecen igual. Revisé la gramática original regla por regla y esto fue lo que cambié.

Problema que encontré	Qué hice	Para qué sirve
Recursión por la izquierda. Revisé todas las reglas y no hay.	Dejé las repeticiones siempre al final de cada regla.	Una regla que empieza llamándose a sí misma haría que la función se llamara sin parar.
En la lista, las dos opciones empezaban con item_lista.	La partí en lista y lista_resto.	El parser consume el primer ítem y luego mira si viene otra viñeta.
En item_lista, las dos opciones empezaban con VIÑETA y texto.	Dejé un solo camino y moví la decisión a sublista_opt.	La decisión se toma con INDENT, que no se parece a nada más.
En la cita, las dos opciones empezaban igual.	La partí en cita, cita_resto y linea_cita.	Es la misma idea que en la lista.
Un párrafo podía quedar vacío porque en_linea puede ser vacío.	Ahora un párrafo son una o más líneas, y cada una empieza con un elemento.	Un párrafo nunca queda vacío y acepta varias líneas.
Dentro de una negrita, otro doble asterisco no se sabía si cerraba o abría.	Dentro de una negrita no se permite otra negrita directa, y dentro de una cursiva no se permite otra cursiva directa.	Si el parser está dentro de una negrita y ve NEGRITA, solo puede ser el cierre.
La cita podía contener cualquier bloque adentro.	Una línea de cita lleva solo texto en línea, y los niveles salen de repetir CITA.	Es una simplificación a propósito y es lo que se pide con > y >>.
Faltaban FIN_LINEA, EOF, la línea en blanco como bloque y el cierre del bloque de código.	Los agregué.	Sin ellos no se podían reconocer textos válidos con líneas en blanco.

4.2 Un conflicto que quedó y cómo lo resuelvo

Quedaron tres reglas con una duda parecida: parrafo_resto, lista_resto y cita_resto. En las tres el parser puede seguir con otra línea del mismo elemento o darlo por terminado, porque lo que viene después puede empezar con el mismo token. Es el problema conocido del else que no se sabe a cuál if pertenece.

La regla que aplica el parser es sencilla: si el token que viene puede continuar el elemento actual, lo continúa. Solo termina cuando ve algo que no puede continuarlo, que puede ser una línea en blanco, un DEDENT, el fin del archivo o el inicio de otro tipo de bloque. Esto coincide con Markdown, donde dos líneas de texto seguidas son el mismo párrafo y, para separarlas, hay que dejar una línea vacía.

5. El analizador sintáctico

5.1 La idea general

El parser tiene una función por cada parte de la gramática. La función que lee un encabezado, la que lee una lista, la que lee una cita y así con las demás. Cada una sabe qué token espera primero y qué otras funciones tiene que llamar para el resto de su regla.

Empieza por la función del documento, que llama a la de los bloques, que a su vez decide a cuál de las funciones de bloque pasar. Cada función consume los tokens que le tocan, crea un nodo del árbol y le cuelga sus hijos.

5.2 Cómo decide qué hacer

En cada punto mira únicamente el token que viene, sin consumirlo, y según cuál sea escoge el camino.

Si el token que viene es	El parser entra a
NUMERAL	Encabezado
VIÑETA	Lista
CITA	Cita
CERCA_CODIGO	Bloque de código
REGLA_HORIZONTAL	Regla horizontal
SALTO_PARRAFO	Lo consume y sigue
TEXTO, NEGRITA, CURSIVA o CODIGO_INLINE	Párrafo
EOF	Termina el documento

5.3 La recursión en las listas

Esta es la parte más importante. Un ítem de lista lee su viñeta, su texto y su fin de línea. Después mira el siguiente token. Si es un INDENT, significa que viene una sublista, así que llama otra vez a la función de lista, que lee sus propios ítems. Cuando esa sublista termina aparece un DEDENT, que cierra el nivel y devuelve el control al ítem de afuera.

Como la función de lista se llama a sí misma por medio del ítem, no hay un límite fijo de niveles. Tres niveles son simplemente tres llamadas una dentro de otra.

5.4 La recursión en las citas

Cada símbolo > es un token aparte. Una línea de cita lee su primer CITA y entra al cuerpo. Si lo que viene es otro CITA, el cuerpo se llama a sí mismo y suma un nivel más. Cuando lo que viene ya no es un CITA, lee el texto en línea y el fin de línea y se detiene. El nivel de la cita sale de contar cuántas veces se llamó a sí mismo, y por eso > > y >> producen lo mismo.

5.5 El texto dentro de una línea

La negrita y la cursiva se leen con funciones que se llaman entre sí. Una negrita puede tener una cursiva adentro y una cursiva puede tener una negrita adentro, pero no una del mismo tipo. Eso evita la duda de si un doble asterisco abre o cierra. El código en línea no admite nada adentro, porque lo que hay entre los acentos graves es un solo literal.

6. El árbol

El árbol tiene dos usos. Por un lado está el árbol del documento que construye el parser, con nodos como encabezado, lista, ítem o cita, y es el que recorre el generador para producir el HTML. Por otro lado está el árbol de derivación, que muestra cada regla de la gramática que se aplicó, y es el que se exporta al archivo arbol.dot.

En el archivo, los nodos de las partes de la gramática y los de los tokens llevan colores distintos. Esto es un ejemplo parcial para un documento que solo tiene el encabezado Hola:

```
digraph ArbolDerivacion {
  rankdir=TB;
  node [shape=box, style=filled, fontname="Arial"];
  n0 [label="<documento>", fillcolor="#2E75B6", fontcolor=white];
  n1 [label="<bloques>", fillcolor="#D6EAF8"];
  n2 [label="<bloque>", fillcolor="#D6EAF8"];
  n3 [label="<encabezado>", fillcolor="#D6EAF8"];
  n4 [label="NUMERAL", fillcolor="#F9E79F"];
  n5 [label="<en_linea>", fillcolor="#D6EAF8"];
  n6 [label="<elemento>", fillcolor="#D6EAF8"];
  n7 [label="TEXTO", fillcolor="#F9E79F"];
  n8 [label="<en_linea>", fillcolor="#D6EAF8"];
  n9 [label="ε", fillcolor="#F9E79F"];
  n10 [label="FIN_LINEA", fillcolor="#F9E79F"];
  n11 [label="<bloques>", fillcolor="#D6EAF8"];
  n12 [label="ε", fillcolor="#F9E79F"];
  n13 [label="EOF", fillcolor="#F9E79F"];
  n0 -> n1; n0 -> n13;
  n1 -> n2; n1 -> n11;
  n2 -> n3;
  n3 -> n4; n3 -> n5; n3 -> n10;
  n5 -> n6; n5 -> n8;
  n6 -> n7;
  n8 -> n9;
  n11 -> n12;
}
```

Para ver el dibujo se pasa este código por Graphviz. En el documento de casos de prueba se incluyen árboles de los ejemplos más completos.

7. Errores y cómo se recupera el programa

7.1 Los siete errores

Error	Quién lo detecta	Cuándo ocurre	Mensaje
NEGRITA_SIN_CERRAR	Lector	Se abrió ** y la línea terminó sin cierre.	Negrita sin cerrar iniciada en línea L, columna C
CURSIVA_SIN_CERRAR	Lector	Se abrió * en posición de énfasis y no se cerró.	Cursiva sin cerrar iniciada en línea L, columna C
CODIGO_INLINE_SIN_CERRAR	Lector	Se abrió un acento grave y no se cerró.	Código en línea sin cerrar iniciado en línea L, columna C
BLOQUE_CODIGO_SIN_CERRAR	Lector	Se abrió la cerca y se llegó al final del archivo sin cierre.	Bloque de código sin cerrar iniciado en línea L
ENCABEZADO_INVALIDO	Lector	Hay más de seis # al inicio de línea.	Encabezado inválido en línea L: máximo 6 niveles
INDENTACION_INVALIDA	Parser	Un ítem sube más de un nivel de sangría de golpe.	Indentación inválida en línea L, columna C
CITA_MAL_FORMADA	Parser	Un > no está seguido de un espacio ni de otro >.	Cita mal formada en línea L, columna C

7.2 La estrategia de recuperación

El programa nunca se detiene en el primer error. Cuando encuentra uno, hace tres cosas: lo anota en el gestor de errores con su tipo, su mensaje y su posición, descarta lo mínimo necesario para no quedarse atascado y sigue leyendo desde un punto seguro. A esto se le llama modo pánico.

Para los delimitadores sin cerrar, el error se reporta en el lugar donde se abrió, no donde terminó la línea, porque ahí es donde el usuario tiene que corregir. Después, el resto de esa línea se trata como texto común y el análisis sigue en la línea siguiente. Para los demás errores, el punto seguro es el siguiente token que sí tenga sentido en ese lugar, por ejemplo la siguiente línea o la siguiente viñeta.

El resultado es que en una sola pasada el usuario recibe todos los problemas del archivo.

7.3 Un ejemplo de cadena inválida

Tomo este texto:

```
Esto es **importante
```

Los tokens son TEXTO, NEGRITA, TEXTO, FIN_LINEA y EOF. El parser entra a un párrafo, lee el primer TEXTO, y al ver NEGRITA entra a la regla de negrita. Consume la apertura y el TEXTO

que sigue, pero cuando espera más contenido o el cierre se encuentra con FIN_LINEA. Dentro de una negrita solo puede venir texto, cursiva, código en línea o el cierre, y FIN_LINEA no es ninguna de esas cosas. La gramática exige que toda negrita se cierre antes de que acabe la línea, así que el texto se rechaza y se reporta el error en el lugar donde se abrió.

8. Los reportes

Cuando el análisis termina sin errores se habilitan los botones de reportes y se generan estos archivos en la carpeta reportes.

Archivo	Qué contiene	Cómo se arma
Documento	El texto convertido a HTML con encabezados, listas, citas, negritas, cursivas y código.	Se recorre el árbol y cada tipo de nodo se traduce a su etiqueta.
Estadísticas	Cuántas palabras, párrafos, encabezados por nivel, listas, citas y bloques de código hay, y qué tan profundo llegan las listas y las citas.	Se recorre el árbol contando nodos y midiendo la profundidad.
Tabla de contenido	Un índice con los encabezados, con más sangría mientras más bajo es el nivel, y enlaces a cada sección.	Se toman solo los nodos de encabezado y se les pone un ancla en el documento.
arbol.dot	El árbol de derivación para Graphviz.	Se recorre el árbol y por cada nodo se escribe una línea con su color.

El documento convierte los símbolos propios del HTML en texto seguro antes de escribirlos, para que un texto como una desigualdad no se confunda con una etiqueta. El contenido de los bloques de código se escribe tal cual, sin interpretar nada.

9. Decisiones de diseño y por qué las tomé

Un token por cada símbolo de cita. El enunciado propone un solo token con la cantidad de símbolos. Yo preferí un token por cada uno, para que el nivel de la cita salga solo de la recursión de la gramática y para que las dos formas de escribir una cita den los mismos tokens.

Citas que solo llevan texto. Una cita podría contener listas o encabezados, pero la simplifiqué a propósito. Con eso la gramática queda más limpia y cubre lo que se pide, que son citas de dos niveles.

Negrita y cursiva sin repetirse. Impedir que una negrita contenga otra negrita directa quita la duda de si un doble asterisco abre o cierra. A cambio, no se permite escribir una negrita dentro de otra, lo cual en la práctica casi no se usa.

Pila de sangría en el lector y no en el parser. Me pareció más ordenado que el lector entregue INDENT y DEDENT ya listos, igual que hace Python con sus bloques. El parser solo ve la estructura y no tiene que contar espacios.

Generar el HTML desde el árbol. Es obligatorio por el enunciado, pero también es lo más cómodo. Una vez que el árbol está bien armado, traducirlo es recorrerlo, con la misma

recursión que usó el parser.

Un gestor de errores aparte. Así el lector y el parser no se preocupan por cómo se muestran los errores. Solo los reportan y la interfaz los lee de un solo lugar.

10. Pruebas y límites

El proyecto trae cuatro scripts de prueba en la carpeta tests, que cubren el lector, el parser, el HTML y los reportes. Se ejecutan desde la carpeta del proyecto con estos comandos:

```
python -m tests.prueba_lexico
python -m tests.prueba_sintactico
python -m tests.prueba_html
python -m tests.prueba_reportes
```

También hay un generador que arma el documento de casos de prueba con trece casos, que incluyen un documento válido, listas y citas anidadas, errores de formato y de sangría, un archivo vacío y casos extremos.

Hay dos límites que conviene tener claros. Primero, los scripts comprueban resultados, pero no revisan cómo se ve la interfaz, así que esa parte se verifica a mano con capturas. Segundo, el autómata del lector no existe como una tabla de estados dentro del código. Lo que se muestra en este manual es una descripción de las decisiones que toma, no una estructura que el programa use.